

Mock School System - Attacks and Countermeasures Life Cycle

Man-in-the-Middle Attack

Research Assistant & Author Brian Alexis Almaguer
Principle Investigator Dr. Jun Dai

Background

You have access to and manage a school system and website to track students, teachers, and grades. This web based application hosts information that is considered Personally Identifying Information (PII) that can be accessed by a one-to-one (1:1) relationship, respectively, by the owner of the information; Teachers and Administrators, however, have a one-to-many (1:*) relationship to the data since they manage the records and there updates. The Web application is a three-layer application; it is coded in PHP, severed to the web using Apache, and uses MySQL to store the user data. The site has a log-in that creates one of two 1:1 or 1:* views for the data. The 1:* view allows for the data to be edited in an editing view. All of this is designed and needed by the school whose system you manage.

Scenario

Every day since finals week the web application has been taking 30 to 180 seconds to load on every teacher's computer. The network on closer inspection for multiple edge routers is being flooded with ARP requests purposely flooding the network. All the signs are pointing this to be a Man-in-the-Middle attack; someone in the network is desperately trying to intercept communications, but you don't know for what data.

SSDLC

The Secure Software Development Life Cycle (SSDLC), is what we are going to follow to since it helps organizations develop secure software. SSDLC is also what we are going to follow, since software, made with SSDLC, is developed with security considerations at every stage of the development process. More can be read here¹ in the wonderful Hackerone article.

Planning

1. Step 1 - Recreate the bug or attack

In the planning phase of the SSDLC, since we already have a preexisting product, we are going to need to recreate the bug or, in this case, the Man-in-the-Middle Attack. Since this attack is looking for data we know we need a good all encompassing recreation. To do this, we will use a special operating system with the software we need to recreate and track all network traffic. The following are the command we will use in a generic form.

```
1 // use this to MITM the traffic both ways
2 arpspoof -t <target> <gateway>
3 arpspoof -t <gateway> <target>
4 // use this to view live traffic
5 webspay -i <interface>
6 // use this to save all traffic to a file
7 tcpdump -i <interface> -w <file> -s 0 tcp
8 // use this to view the saved traffic
9 tcpick -C -yP -r <file>
```

The the operating system that has these program ready or ready for install is Kali Linux. The program arpspoof operates by intercepting and manipulating Address Resolution Protocol (ARP) traffic on a local area network (LAN). webspay is a program that allows live network packet data to be displayed for the user

¹<https://www.hackerone.com/knowledge-center/what-ssdlc-secure-software-development-life-cycle>

when a network interface card (NIC) is actively being used. tcpdump is a program that allows us to save our TCP traffic to a file, while tcpick allows us to display that same file to a terminal with color.

2. Step 2 - Risk Assessment

Not all attacks/bugs actually need a Risk Assessment, since a large majority of them have been categorized on site such as OWASP² (Open Worldwide Application Security Project). In this case, it is really clear that an indef risk assessment is needed since we don't know which data our attacker is after since they are trying to do Man-in-the-Middle attack.

3. Step 3 - Mitigation

Since the Attack Recreation and Risk Assessment defined exactly what we are dealing with, we now need to research and catalog what is out there for mitigation's. A great starting point is to look at articles that OWASP has exactly like following³.

Requirements and Analysis

1. Step 1 - Analysis

Once the research has been completed on the attack and its recreation, the underlying technology or code needs to be connected to your work. Doing this makes the next step easier to do.

2. Step 2 - Requirements

This step can be fleshed out until the connection between the attack/bug is connected to the technology or code. That connection essentially writes your requirements out for you; it is your job to expand on those connections into simpler explanations and ideas.

For our specific example project, our attack does not have a clear goal. The attack points to this problem being a Man-in-the-Middle attack, but it could as well be a Denial of Service. Simulating a Man-in-the-Middle attack generates data to identify any data that might not be desirably leaked. Because of this, our requirements are as follows:

- (a) Collect all TCP data
- (b) Search for usernames
- (c) Search for passwords
- (d) Search for any PII

Design and Prototyping

1. Step 1 - Design

Not all designs need to be implemented in the product to work during this phase. Some designs can be generated at a high level without actually writing code. In our case, we need to explore network traffic.

2. Step 2 - Prototyping

Like Design, prototyping does not need to to be implemented in the product; prototypes can be made outside of the larger product to show proof of designs in that way the product does not receive any unguided development changes.

Development

1. Step 1 - Version Control or Backups

In Development, the very first thing you do is back up or version control your original program and settings before testing your prototypes on the product. Data should also be backed up just in case of a full system breakdown.

2. Step 2 - Run/Create a Non-Production Environment

Part of development is testing; true product testing can be done when you have a mirrored live or local product. In our case, we have a code ready to be edited live to see the change made live.

²<https://owasp.org/about/>

³https://owasp.org/www-community/attacks/Manipulator-in-the-middle_attack

3. Step 3 - Integration

This step involves or effort, as you now have to integrate your design and prototype into the mock product before attempting a simulated attack. This step can happen multiple times over multiple days if we are going to make a countermeasure for a larger or more complicated attack.

Deployment

Deployment is the most important step as a maintainer since your web application will have to be down preventing any of your uses from accessing the data they need. Depending on the strategy used in the organization, a change process would be needed before making any changes. In the end, the process always follows an unfreeze-change-freeze flow.

Maintenance

Common regular web application maintenance should include performing daily backups of both application code and database content, monitoring server logs for errors or suspicious activities, and periodically updating the security of all dependencies and frameworks. In this scenario, it is imperative that backups of the code are saved.